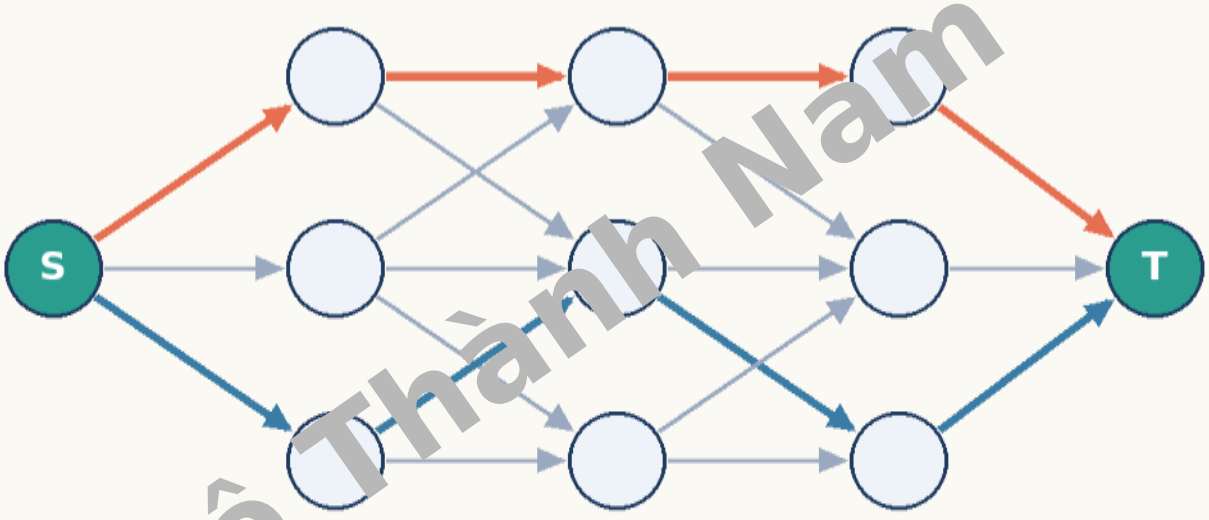
TUYỂN TẬP BÀI TẬP LUYỆN THI TIN HỌC

25 bài CSES Problem Set, dịch tiếng Việt

KĨ THUẬT NÂNG CAO



Lê Thành Nam

hai lộ trình rời nhau có tổng chi phí nhỏ nhất (luồng chi phí nhỏ nhất)

Gặp nhau ở giữa · Bitset | Treap ngầm | Cầu, khớp, DSU nâng cao | Tối ưu quy hoạch động | FFT | Luồng chi phí nhỏ nhất

Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu

LÊ THÀNH NAM

Tổng hợp và biên soạn

MỤC LỤC

Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu

Giới thiệu 2

Chương 1. Chia đôi tập hợp và bitset 3

Bài 1. Gặp nhau ở giữa *(CSES 1628)* 4
Bài 2. Khoảng cách Hamming *(CSES 2136)* 8
Bài 3. Kiểm tra lưới con bốn góc *(CSES 3360)* 11
Bài 4. Đếm lưới con bốn góc *(CSES 2137)* 16
Bài 5. Các đỉnh tới được *(CSES 2138)* 20
Bài 6. Truy vấn đường đi *(CSES 2143)* 24

Chương 2. Cây Treap ngầm 28

Bài 7. Cắt và dán *(CSES 2072)* 29
Bài 8. Đảo ngược đoạn con *(CSES 2073)* 33
Bài 9. Đảo ngược và tính tổng *(CSES 2074)* 37

Chương 3. Kĩ thuật nâng cao trên đồ thị 41

Bài 10. Những con đường thiết yếu *(CSES 2076)* 42
Bài 11. Những thành phố thiết yếu *(CSES 2077)* 47
Bài 12. Đồ thị con Euler *(CSES 2078)* 51
Bài 13. Truy vấn đường mới *(CSES 2101)* 55
Bài 14. Liên thông động *(CSES 2133)* 59

Chương 4. Tối ưu quy hoạch động 63

Bài 15. Trò chơi quái vật I *(CSES 2084)* 64
Bài 16. Trò chơi quái vật II *(CSES 2085)* 68
Bài 17. Bình phương đoạn *(CSES 2086)* 72
Bài 18. Nhà và trường học *(CSES 2087)* 77
Bài 19. Phép chia Knuth *(CSES 2088)* 81

Chương 5. Biến đổi Fourier nhanh 85

Bài 20. Táo và chuối *(CSES 2111)* 86
Bài 21. Vị trí các bit 1 *(CSES 2112)* 90
Bài 22. Xử lí tín hiệu *(CSES 2113)* 94

Chương 6. Luồng chi phí nhỏ nhất 98

Bài 23. Giao bưu kiện *(CSES 2121)* 99
Bài 24. Phân công công việc *(CSES 2129)* 104
Bài 25. Các lộ trình phân biệt II *(CSES 2130)* 109

Lê Thành Nam

GIỚI THIỆU

Tài liệu này dành cho ai

Tài liệu gồm toàn bộ 25 bài của mục Advanced Techniques trong CSES Problem Set (cses.fi/problemset), được sắp lại thành 6 chương theo kĩ thuật: *gặp nhau ở giữa và bitset, cây Treap ngầm, kĩ thuật nâng cao trên đồ thị (cầu, khớp, DSU nâng cao), tối ưu quy hoạch động (bao lồi đường thẳng, cây Li Chao, chia để trị, tối ưu Knuth), biến đổi Fourier nhanh, và luồng chi phí nhỏ nhất*. Tài liệu phù hợp với học sinh đã nắm vững các tập Sắp xếp, Quy hoạch động và Đồ thị, đang luyện thi học sinh giỏi cấp tỉnh, cấp quốc gia và các kì thi tương đương.

Mỗi bài gồm hai phần. Đề bài được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc, kèm bảng subtask do tài liệu bổ sung. Hướng dẫn giải đi qua 4 bước tư duy; mỗi bước có code chạy được, có hình minh họa, bài nào cũng kết thúc bằng bảng so sánh các tầng lời giải đo thật trên máy. Ở những bài có cách làm "tự nhiên" nhưng sai (tham lam trong bài luồng), tài liệu chỉ rõ phản ví dụ.

4 bước tư duy giải một bài

Bước	Nội dung
Ngây thơ	Ý tưởng tự nhiên nhất, làm đúng như đề mô tả, có code và độ phức tạp.
Bế tắc	Biểu đồ thời gian đo thật, chỉ ra chỗ chậm, nút cổ chai, kết thúc bằng một câu hỏi then chốt.
Phá bế tắc	Nhận xét hoặc tính chất giúp vượt qua chỗ tắc, có ví dụ và hình minh họa.
Tối ưu	Code cuối cùng, độ phức tạp và các lưu ý: kiểu dữ liệu, tràn số, đọc ghi nhanh.

Subtask và bảng so sánh các tầng

Đề CSES không chia subtask. Tài liệu bổ sung cho mỗi bài ba mức: Subtask 1 (30%) là ràng buộc nhỏ để cách ngây thơ lấy điểm, Subtask 2 (30%) và Subtask 3 (40%) là ràng buộc gốc của CSES hoặc ràng buộc mở rộng (thường gấp 3 tới 10 lần). Ràng buộc mở rộng được chọn để tách bạch những lời giải cùng đạt ràng buộc gốc: ví dụ DFS đệ quy và không đệ quy, Bellman-Ford và Dijkstra, bitset và FFT.

Cuối mỗi bài là bảng so sánh: mỗi tầng code chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm. Một tầng chỉ được điểm của subtask nếu cho kết quả đúng trong không quá 1 giây, dùng không quá 512 MB bộ nhớ và 8 MB ngăn xếp (ô "sai" là chương trình cho kết quả sai, ô "lỗi" là chương trình bị dừng vì tràn bộ nhớ hoặc tràn ngăn xếp). Thời gian đo bằng g++ 13 (tùy chọn -O2) trên máy chủ Linux, tính cả đọc và ghi dữ liệu, lấy lần nhanh hơn trong hai lần chạy; trên máy chấm khác các con số có thể chênh lệch đôi chút.

Sử dụng tài liệu

Với mỗi bài, hãy tự suy nghĩ ít nhất 20–30 phút trước khi đọc hướng dẫn. Khi đọc, dừng lại ở câu hỏi then chốt cuối phần **Bế tắc** và tự trả lời trước khi xem phần **Phá bế tắc**.

Mọi code trong tài liệu đã được biên dịch, chạy đúng ví dụ của đề, và đối chiếu với lời giải tham chiếu (hoặc bộ kiểm tra đáp án) trên 300 test ngẫu nhiên. Các bài FFT còn được đối chiếu trên test lớn nhất với phép nhân số nguyên lớn chính xác; các bài luồng được đối chiếu với thư viện networkx và scipy.

Lê Thành Nam

Chương 1. Chia đôi tập hợp và bitset

Trọng tâm chương

Nhiều bài có lời giải vét cạn rõ ràng nhưng quá chậm một chút: 2^{40} tập con, n^3 phép tính với n = 3 000, hay n lần loang đồ thị. Chương này giới thiệu hai cách "làm nhỏ" công việc: gặp nhau ở giữa (chia đôi tập hợp, giải từng nửa rồi ghép lại) và bitset (gói 64 giá trị đúng/sai vào một số nguyên, xử lí chúng bằng một lệnh).

Kiến thức cần nhớ

- Gặp nhau ở giữa. 2^n thành $2 \cdot 2^{(n/2)}$: liệt kê từng nửa, ghép bằng sắp xếp + hai con trỏ (hoặc tìm kiếm nhị phân).
- Phép toán bit. XOR cho các vị trí khác nhau, AND cho các vị trí cùng bằng 1, popcount đếm số bit 1.
- #pragma GCC target("popcnt"). Cho phép dùng lệnh đếm bit của CPU; đặt sau các dòng #include.
- Bitset. bitset<N> hoặc mảng số 64 bit; phép OR/AND trên cả dãy nhanh hơn gấp 64 lần vòng lặp thường.
- Nguyên lí chuồng bồ câu. Nếu số trạng thái bị chặn, thuật toán [illegible] đúng trước khi một trạng thái lặp lại.

Các bài trong chương

Bài	Tên bài	Mã CSES	Kĩ thuật chính
1	Gặp nhau ở giữa	1628	Gặp nhau ở giữa
2	Khoảng cách Hamming	2136	XOR + popcount
3	Kiểm tra lưới con bốn góc	3360	Chuồng bồ câu
4	Đếm lưới con bốn góc	2137	Bitset
5	Các đỉnh tới được	2138	Tô-pô + bitset
6	Truy vấn đường đi	2143	SCC + bitset

Lê Thành Nam

Bài 1. Gặp nhau ở giữa

CSES 1628 – Meet in the Middle Thời gian: 1 giây · Bộ nhớ: 512 MB
Đề gốc: https://cses.fi/problemset/task/1628

Đề bài

Cho dãy n số nguyên dương $t_1, t_2, \ldots, t_n$ và số nguyên x. Hãy đếm số cách chọn một tập con của dãy (các phần tử ở vị trí khác nhau được coi là khác nhau) sao cho tổng các phần tử được chọn bằng x.

Dữ liệu vào

Dòng đầu chứa hai số nguyên n và x.

Dòng thứ hai chứa n số nguyên $t_1, t_2, \ldots, t_n$.

Ràng buộc chung: $1 \le x, t_i \le 10^9$.

Kết quả

Một số nguyên là số cách chọn.

Ví dụ

Dữ liệu vào	Kết quả
4 5 1 2 3 2	3

Giải thích ví dụ

Ba cách: $t_2 + t_3 = 5$, $t_3 + t_4 = 5$ và $t_1 + t_2 + t_4 = 5$.

Subtask

Subtask	Ràng buộc	Điểm
Subtask 1	$n \le 2$	30
Subtask 2	$n \le 30$	30
Subtask 3	$n \le 40$ (ràng buộc gốc)	40

Lê Thành Nam

Hướng dẫn giải

NGÂY THƠ Thử mọi tập con

Mỗi phần tử có hai lựa chọn: Chọn hoặc không chọn. Viết hàm đệ quy go(i, sum): ở phần tử i, thử không chọn rồi thử chọn; tới cuối dãy thì kiểm tra sum có bằng x không. Vì các số đều dương, khi sum đã vượt x thì dừng nhánh đó ngay.

Ví dụ: Với dãy 1 2 3 2 và x = 5 có ba cách: $t_2 + t_3$, $t_3 + t_4$ và $t_1 + t_2 + t_4$.

Code. Tầng 1. Thử mọi tập con

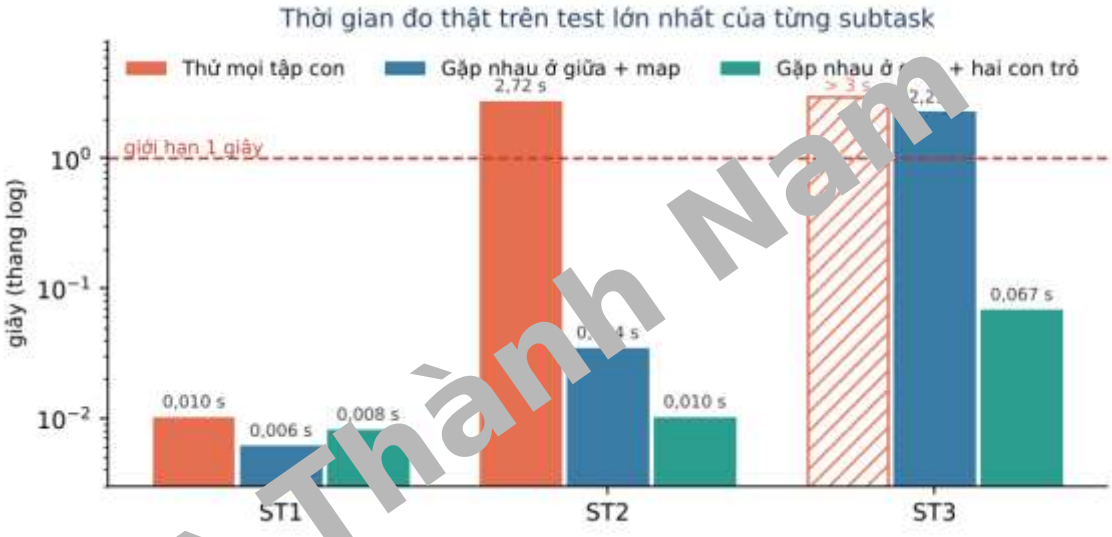
```
#include <bits/stdc++.h>
using namespace std;
int n; long long x, t[45], cnt = 0;
// Thử mọi tập con: ở phần tử i, hoặc không chọn, hoặc chọn
void go(int i, long long sum) {
    if (sum > x) return;                    // các số đều dương: vượt x thì dừng
    if (i == n) { if (sum == x) cnt++; return; }
    go(i + 1, sum);
```

```
    go(i + 1, sum + t[i]);
}
int main() {
    scanf("%d %lld", &n, &x);
    for (int i = 0; i < n; i++) scanf("%lld", &t[i]);
    go(0, 0);
    printf("%lld\n", cnt);
}
```

Độ phức tạp: $O(2^n)$.

BẾ TẮC 2^{40} tập con

Với n = 30 đã có $2^{30} \approx 10^9$ tập con: Mất 2,7 giây; với n = 40 là $2^{40} \approx 10^{12}$ tập con, không thể duyệt hết (Hình 1.1). Việc cắt nhánh khi vượt x chỉ giúp được khi các số lớn so với x. Câu hỏi then chốt: 2^{40} thì quá lớn, nhưng $2^{20} \approx 10^6$ thì rất nhỏ. Có cách nào chỉ cần duyệt các tập con của một nửa dãy không?



Hình 1.1. Thời gian đo thật của ba tầng lời giải.

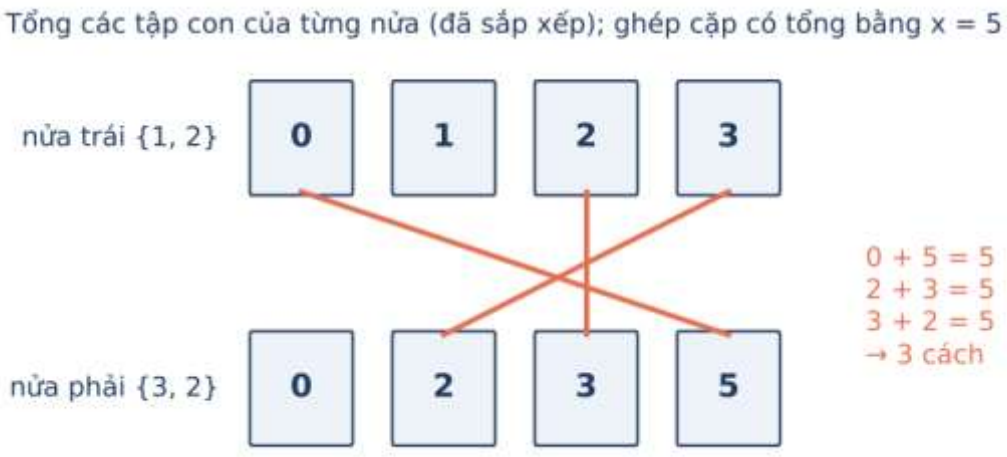
PHÁ BẾ TẮC Phá bế tắc: Gặp nhau ở giữa

Chia dãy thành hai nửa. Mỗi tập con của cả dãy là một tập con L của nửa trái ghép với một tập con R của nửa phải, và tổng của nó là sum(L) + sum(R). Vậy số cách bằng số cặp (L, R) với sum(L) + sum(R) = x.

Liệt kê tổng của mọi tập con nửa trái (2^{20} số) và đếm bằng map: cntL[s] là số tập con có tổng s. Sau đó với mỗi tập con nửa phải có tổng s, cộng thêm cntL[x - s] (Hình 1.2). Thay vì 2^{40} ta chỉ làm $2 \cdot 2^{20}$ bước, mỗi bước một thao tác trên map.

Với n = 30 chỉ mất 0,03 giây. Nhưng với n = 40, map phải chứa khoảng một triệu khóa; mỗi thao tác map mất O(log) và truy cập bộ nhớ rải rác, nên tổng cộng mất 2,2 giây.

Lê Thành Nam



Hình 1.2. *Gặp nhau ở giữa với ví dụ: t = 1 2 3 2, x = 5.*

Code. Tầng 2. Gặp nhau ở giữa + map

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; long long x;
    scanf("%d %lld", &n, &x);
    vector<long long> t(n);
    for (auto &v : t) scanf("%lld", &v);
    int h = n / 2;                          // nửa trá[0..h-1], nửa phải: t[h..n-1]
    // Đếm số tập con của nửa trái theo từng tổng
    map<long long, int> cntL;
    for (int mask = 0; mask < (1 << h); mask[illegible]
        long long s = 0;
        for (int i = 0; i < h; i++) if [illegible]as[illegible] >> i & 1) s += t[i];
        cntL[s]++;
    }
    // Với mỗi tập con của nửa phải có tổng s, cần nửa trái có tổng x - s
    long long ans = 0;
    int r = n - h;
    for (int mask = 0; mask < (1 << r); mask++) {
        long long s = 0;
        for (int i = 0; i < r; i++) if (mask >> i & 1) s += t[h + i];
        auto it = cntL.find(x - s);
        if (it != cntL.end()) ans += it->second;
    }
    printf("[illegible]d\n", ans);
}
```

Lê Thành Nam

TỐI ƯU Hai dãy tổng đã sắp xếp + hai con trỏ

Bỏ map: Sắp xếp tổng các tập con của mỗi nửa thành dãy A (tăng dần) và B (tăng dần), rồi đếm số cặp A[i] + B[j] = x bằng hai con trỏ: i đi từ đầu A, j đi từ cuối B. Tổng nhỏ hơn x thì tăng i, lớn hơn x thì giảm j; bằng x thì đếm cả nhóm các giá trị bằng nhau ở hai phía (ca · cb cặp) rồi bỏ qua cả hai nhóm.

Cách thứ nhất để có dãy tổng đã sắp xếp: Sinh 2^{20} tổng rồi gọi sort, mất O(2^(n/2) · n). Cách thứ hai, nhanh hơn: Thêm dần từng phần tử v. Nếu S là dãy tổng (đã sắp xếp) của các phần tử trước, thì dãy mới gồm S và S + v; cả hai đều tăng dần nên trộn (merge) chúng trong thời gian tuyến tính. Tổng thời gian là 1 + 2 + 4 + … + 2^(n/2) < 2 · 2^(n/2).

Với n = 40 chạy 0,07 giây, nhanh hơn bản dùng map khoảng 30 lần.

Code. Tầng 3. Gặp nhau ở giữa + hai con trỏ

```
#include <bits/stdc++.h>
using namespace std;
// Sinh tổng của mọi tập con của a, đã sắp tăng dần, trong O(2^|a|):
```